

FinTech Nova

Manual de Ejecucion, Monitoreo y Automatizacion (Windows)

1. Introduccion

Este manual describe los pasos necesarios para desplegar, ejecutar y monitorear la API de FinTech Nova en un entorno Windows de manera local.

La aplicacion consta de un backend desarrollado en FastAPI (Python) que evalua el riesgo crediticio y registra solicitudes en una base de datos SQLite. Adicionalmente, cuenta con una suite de herramientas de automatizacion para backups, monitoreo de recursos y analisis de logs de seguridad.

Nota de compatibilidad: Los scripts Bash (.sh) originales incluidos en el proyecto requieren entornos Linux. Para facilitar la ejecucion directa en Windows sin dependencias complejas como Docker o WSL, hemos creado equivalentes en scripts de Python (.py) nativos que realizan las mismas tareas de manera multiplataforma.

2. Requisitos Previos

Para ejecutar el proyecto necesitas:

- Python 3.10 o superior (Se ha verificado Python 3.10.6 instalado en el sistema).
- Pip (Administrador de paquetes de Python).
- Librerias listadas en requirements.txt (se detallan en el Paso 2).

3. Pasos de Instalacion y Ejecucion

Paso 1: Configurar el Entorno Virtual (venv)

Se recomienda crear un entorno virtual para aislar las dependencias del proyecto. Abre una terminal PowerShell en el directorio raiz del proyecto y ejecuta:

```
# 1. Crear el entorno virtual llamado 'venv'
python -m venv venv

# 2. Activar el entorno virtual en PowerShell
.\venv\Scripts\Activate.ps1
```

Paso 2: Instalar Dependencias

Instala las dependencias especificadas utilizando requirements.txt (el archivo ha sido corregido para evitar conflictos de codificacion Unicode en Windows):

```
pip install -r requirements.txt
```

Paso 3: Inicializar la Base de Datos SQLite

La base de datos se inicializa automaticamente al arrancar la API. Tambien puedes crear la base de datos sqlite vacia manualmente ejecutando:

```
python -c "import main; main.init_db()"
```

Paso 4: Arrancar el Servidor FastAPI

Inicia el servidor ASGI de uvicorn en tu localhost:

```
uvicorn main:app --reload --host 127.0.0.1 --port 8000
```

- URL de Estado de la API: <http://127.0.0.1:8000/status>
- Endpoint de Health Check: <http://127.0.0.1:8000/health>
- Documentacion Interactiva (Swagger): <http://127.0.0.1:8000/docs>

4. Herramientas de Automatizacion y Monitoreo

En lugar de usar los archivos .sh que requieren Linux, ejecuta las versiones de Python:

A. Copias de Seguridad de Base de Datos (backup_db.py)

Genera un respaldo comprimido de 'database.db' y mantiene los ultimos 7 dias de copias:

```
python backup_db.py
```

Los archivos se guardaran en la carpeta 'backups/' con el formato 'backup_FECHA.tar.gz'.

B. Monitoreo de Recursos del Sistema (resource_monitor.py)

Verifica el uso de CPU, RAM y espacio en disco en tu maquina local emitiendo alertas si se sobrepasan los umbrales criticos. Requiere la libreria psutil:

```
python resource_monitor.py
```

C. Detector de SQL Injection en Logs (log_analyzer.py)

Analiza un archivo de logs (como server.log) buscando patrones comunes de ataques de inyeccion SQL (ej. OR 1=1, DROP TABLE, UNION SELECT) y genera un reporte:

```
python log_analyzer.py server.log
```

D. Monitoreo Continuo de la API (health_monitor.py)

Consulta periodicamente el endpoint /health de la API y escribe un log historico en health_monitor.log. Util para verificar la disponibilidad del servicio:

```
python health_monitor.py --url http://127.0.0.1:8000/health --interval 10
```